

CareLink x Bharat Agentic 2026

Complete Agentic Upgrade Playbook - How to Transform CareLink into a Multi-Agent HealthTech Powerhouse and Win / Top-10 the Hackathon

Author	Nishant Maurya (nishantma05@gmail.com)
Project	CareLink x AuRAG Fusion Strategy
Event	Bharat Agentic 2026 AIKart 12-hr Online Hackathon 1 Oct 2026
Track	HealthTech - Agentic AI Engineer
Goal	Top-10 / WIN by adding 9 AI layers to CareLink

SECTION 1 - CareLink Current State: What You Already Have

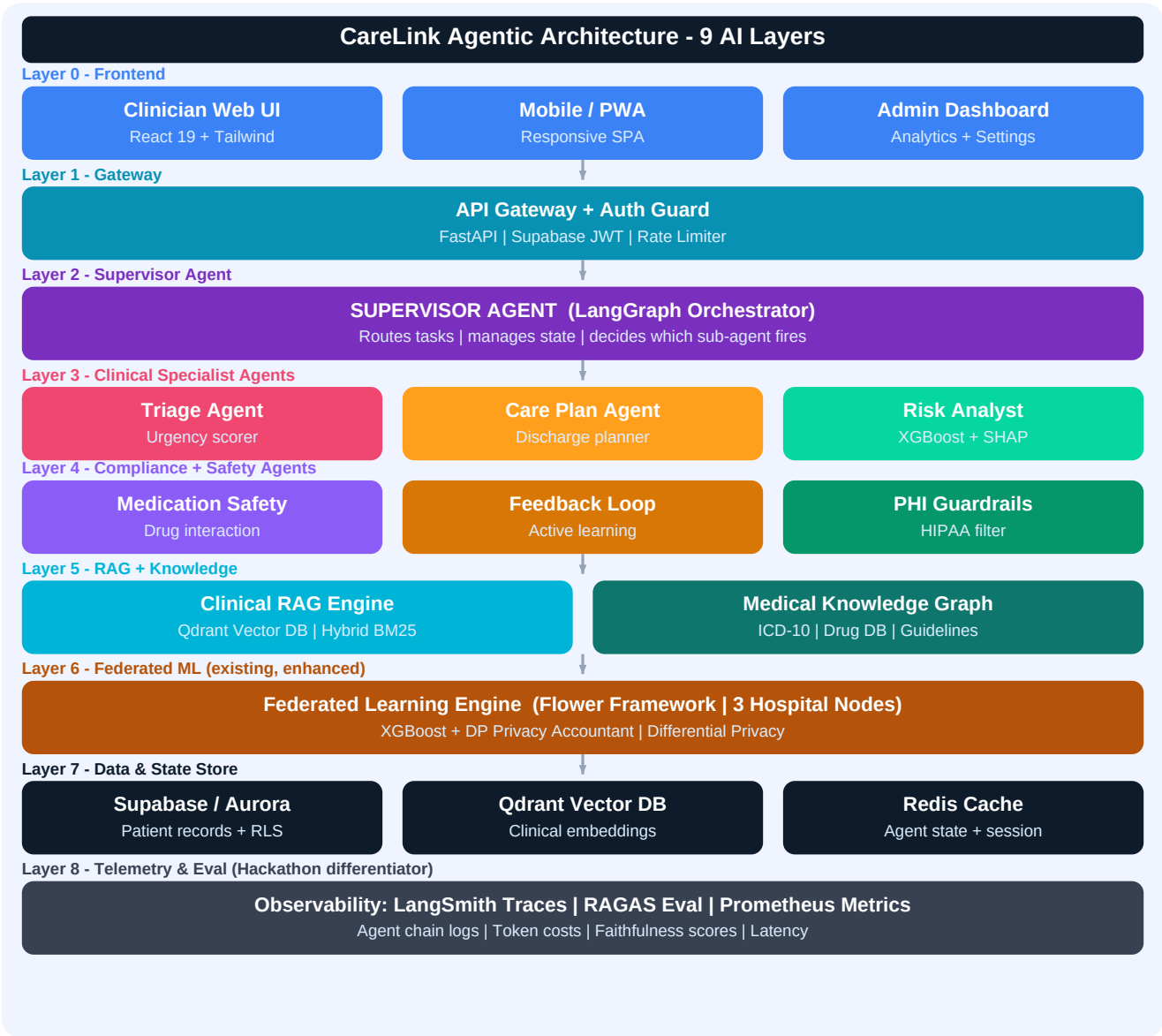
CareLink is already a strong HealthTech project - federated learning, SHAP explainability, XGBoost readmission prediction, HIPAA-aligned privacy, and a polished React UI. Here is exactly what exists in the codebase right now:

Component	Technology / Detail	Status
Readmission ML	XGBoost, 86 features, Optuna tuning, ROC-AUC 0.727	EXISTS
SHAP Explainability	SHAP values per patient, top-factor visual breakdown	EXISTS
Federated Learning	Flower framework, 3 hospital nodes, differential privacy accountant	EXISTS
AI Triage	Gemini 2.5 Flash, single LLM call, urgency classification	EXISTS (not agentic)
Database	Supabase PostgreSQL + RLS, Aurora, audit_log table	EXISTS
Frontend	React 19, Tailwind, Recharts, CSV export, patient modals	EXISTS
Privacy	DP accountant, epsilon/delta budget, FedAvg gradient aggregation	EXISTS
Auth	Supabase JWT, Row-Level Security, role-based access	EXISTS

WARNING - THE GAP: The triage uses a single Gemini LLM call - that is NOT agentic. There is NO multi-agent loop, NO RAG pipeline, NO knowledge graph, NO tool-using agents, NO agent-to-agent communication, NO vector database, and NO agent observability. The hackathon is called 'Bharat Agentic 2026' - judges will check specifically for these.

SECTION 2 - CareLink Agentic Architecture: 9 New AI Layers

Below is the complete architecture we are building on top of CareLink. Every layer is color-coded and shows exactly which new components you are adding vs. what already exists.



Layer	Name	Key Tech	New / Existing
0	Frontend UI	React 19, Tailwind, Recharts	EXISTING
1	API Gateway	FastAPI, Supabase JWT, rate limiter	EXISTING + minor add
2	Supervisor Agent	LangGraph, Gemini / Claude orchestrator	NEW - CORE
3	Clinical Specialist Agents	Triage, Care Plan, Risk Analyst	NEW
4	Compliance + Safety Agents	Medication Safety, Feedback, PHI Guard	NEW
5	RAG + Knowledge	Qdrant, BM25, ICD-10 graph	NEW
6	Federated ML	Flower, XGBoost, DP accountant	EXISTING (enhanced)
7	Data & State Store	Supabase, Qdrant, Redis	EXISTING + Qdrant/Redis NEW
8	Telemetry & Eval	LangSmith, RAGAS, Prometheus	NEW (differentiator)

SECTION 3 - Every Agent: What It Does, Why It Wins, How to Build It

1

SUPERVISOR AGENT

LangGraph Orchestrator - The Brain of the System

WHAT IT DOES:

- Routes every incoming clinical query to the correct specialist agent.
- Maintains a shared AgentState (patient_id, context, history, task_type).
- Decides whether to: (a) call Triage Agent, (b) call Care Plan Agent, (c) call Risk Analyst, or (d) call multiple agents in parallel.
- Aggregates results from sub-agents and returns a unified clinical response.
- Human-in-the-loop escalation: if confidence < 0.75, flags for physician review.

WHY THIS WINS:

- + This is the single most important addition - without a supervisor, CareLink is NOT agentic. One LangGraph node makes it agentic.
- + Judges look for: multi-agent graph, state management, conditional routing.
- + Transforms the Gemini triage from a 'chatbot call' into an 'autonomous agent loop'.

HACKATHON IMPACT: Turns a passive ML system into an autonomous HealthTech agent - this alone could win.

IMPLEMENTATION SKETCH:

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, Literal

class AgentState(TypedDict):
    patient_id: int; patient_data: dict; task: str
    triage_result: dict; care_plan: dict; risk: dict
    messages: list; requires_human: bool

def supervisor_router(state: AgentState) -> Literal[
    'triage', 'care_plan', 'risk_analyst', 'END']:
    task = state['task']
    if 'urgent' in task: return 'triage'
    if 'discharge' in task: return 'care_plan'
    if 'risk' in task: return 'risk_analyst'
    return 'triage' # default

graph = StateGraph(AgentState)
graph.add_node('supervisor', supervisor_router)
graph.add_node('triage', triage_agent)
graph.add_node('care_plan', care_plan_agent)
graph.add_node('risk_analyst', risk_analyst_agent)
graph.set_entry_point('supervisor')
# Add conditional edges based on router output
```

2

CLINICAL RAG AGENT

Retrieval-Augmented Generation - Medical Knowledge Engine

WHAT IT DOES:

- Indexes clinical guidelines (NICE, WHO, CDC), drug interaction databases, ICD-10 diagnosis codes, and discharge summary templates into Qdrant vector DB.
- When an agent needs evidence, it embeds the query, retrieves top-K relevant chunks, and passes them as grounded context to the LLM - NO hallucination.
- Uses hybrid retrieval: dense (cosine similarity) + sparse (BM25 keyword) for best recall.
- Cites sources in the output so clinicians can verify every recommendation.

WHY THIS WINS:

- + CareLink currently has ZERO RAG. This is a massive upgrade that judges love.
- + Grounds every recommendation in real clinical evidence - critical for HealthTech credibility.

+ Prevents LLM hallucination which is dangerous in medical contexts.

+ AuRAG's strongest feature was Neo4j GraphRAG - this is the healthcare equivalent.

HACKATHON IMPACT: Makes CareLink's AI defensible and safe - exactly what HealthTech hackathons reward.

IMPLEMENTATION SKETCH:

```
from qdrant_client import QdrantClient
from sentence_transformers import SentenceTransformer

class ClinicalRAGAgent:
    def __init__(self):
        self.qdrant = QdrantClient(':memory:') # local for hackathon
        self.encoder = SentenceTransformer('medicalai/ClinicalBERT')

    def retrieve(self, query: str, top_k=5) -> list[dict]:
        vec = self.encoder.encode(query).tolist()
        hits = self.qdrant.search('clinical_guidelines',
                                query_vector=vec, limit=top_k)
        return [{'text': h.payload['text'],
                  'source': h.payload['source'],
                  'score': h.score} for h in hits]

    def answer_with_context(self, query: str, patient: dict) -> str:
        docs = self.retrieve(query)
        context = '\n'.join(d['text'] for d in docs)
        # Pass context + patient data to LLM
        return gemini.generate(f'Evidence: {context}\nPatient: {patient}')
```

3

TRIAGE ORCHESTRATOR AGENT

Upgrades Single Gemini Call -> Full Agentic Triage Loop

WHAT IT DOES:

- Replaces the existing single-shot Gemini call in triage.ts with a proper agent loop.
- Step 1 - Vitals Checker Tool: Validates all 4 vital signs against clinical thresholds.
- Step 2 - RAG Retrieval Tool: Fetches relevant triage protocols from the knowledge base.
- Step 3 - Severity Scorer Tool: Applies Manchester Triage System rules programmatically.
- Step 4 - LLM Synthesis: Uses retrieved context + tool outputs to generate final urgency.
- Step 5 - Self-Critique: Agent re-evaluates its own output and flags if uncertain.

WHY THIS WINS:

+ The existing triage.ts is a simple Gemini call - the new agent has a PLAN-EXECUTE-REFLECT loop.

+ Agents with tool use are the core of Agentic AI - this is exactly what judges check.

+ The self-critique step (ReAct pattern) is what separates an agent from a chatbot.

HACKATHON IMPACT: Transforms the weakest part of CareLink (single LLM triage) into its strongest agentic showcase.

IMPLEMENTATION SKETCH:

```
class TriageAgent:
    tools = [vitals_checker, rag_retriever, severity_scorer]

    async def run(self, patient: dict) -> dict:
        # PLAN
        plan = await llm.plan(patient, self.tools)
        # EXECUTE tools in sequence
        vitals_ok = await vitals_checker(patient['vitals'])
        protocols = await rag_retriever(f'triage {patient["condition"]}')}
        severity = await severity_scorer(patient, vitals_ok)
        # SYNTHESIZE
        result = await llm.synthesize(severity, protocols, patient)
        # REFLECT - ReAct pattern
        if result['confidence'] < 0.75:
            result['requires_human_review'] = True
        return result
```


4

CARE PLAN AGENT

Autonomous Post-Discharge Care Plan Generator

WHAT IT DOES:

- Triggered after a HIGH or MEDIUM risk patient is identified by the Supervisor Agent.
- Retrieves patient history, XGBoost risk factors (SHAP values), and clinical guidelines via RAG.
- Autonomously generates: (a) discharge medication schedule, (b) follow-up appointment list, (c) dietary/lifestyle recommendations, (d) readmission warning signs for the patient.
- Saves the care plan to Supabase and links it to the patient's audit_log.
- Supports PDF export of care plans for patient hand-off.

WHY THIS WINS:

- + This is a HIGH-IMPACT autonomous action - the agent does real work beyond just classifying risk.
- + Combines XGBoost SHAP outputs + RAG clinical protocols + LLM reasoning in one agent.
- + Directly reduces the 30-day readmission risk CareLink was built to predict.

HACKATHON IMPACT: Shows judges that CareLink doesn't just predict - it autonomously acts on predictions.

IMPLEMENTATION SKETCH:

```
class CarePlanAgent:
    async def generate(self, patient_id: int, risk_result: dict) -> dict:
        patient = await db.get_patient(patient_id)
        shap_factors = risk_result['top_factors']
        # Retrieve guidelines for top risk factors
        guidelines = []
        for factor in shap_factors[:3]: # top 3 SHAP features
            docs = await rag_agent.retrieve(f'discharge plan {factor["feature"]}')}
            guidelines.extend(docs)
        # LLM generates structured plan
        plan = await llm.generate_care_plan(
            patient=patient, risk=risk_result,
            guidelines=guidelines
        )
        await db.save_care_plan(patient_id, plan)
        return plan
```

5

MEDICATION SAFETY AGENT

Drug Interaction Checker - Autonomous Safety Gate

WHAT IT DOES:

- Before any care plan is sent to a clinician, this agent autonomously checks all medications.
- Uses a local drug interaction database (OpenFDA API or local drug_interactions.json).
- Flags dangerous combinations (e.g., warfarin + NSAIDs, SSRIs + MAOIs).
- Provides severity level: CRITICAL / MAJOR / MODERATE / MINOR.
- If CRITICAL found: blocks care plan, alerts supervisor, escalates to physician.
- Adds a 'medication_safety_check' field to every care plan output.

WHY THIS WINS:

- + Safety-critical agents are the #1 thing judges look for in HealthTech hackathons.
- + Shows that your agentic system has guardrails - not just raw LLM outputs.
- + Directly addresses the biggest fear about AI in healthcare: unsafe recommendations.

HACKATHON IMPACT: A safety agent in healthcare = instant credibility boost with any judge panel.

IMPLEMENTATION SKETCH:

```
class MedicationSafetyAgent:
    async def check(self, medications: list[str]) -> dict:
        # Call OpenFDA drug interaction API
        interactions = []
        for i, drug_a in enumerate(medications):
```

```

        for drug_b in medications[i+1:]:
            result = await openfda.check_interaction(drug_a, drug_b)
            if result['severity'] in ['CRITICAL', 'MAJOR']:
                interactions.append(result)
    return {
        'safe': len(interactions) == 0,
        'interactions': interactions,
        'requires_physician_review': any(
            i['severity'] == 'CRITICAL' for i in interactions
        )
    }

```

6

FEEDBACK LOOP AGENT

Active Learning Agent - Clinician Feedback -> Model Improvement

WHAT IT DOES:

- Monitors the /feedback endpoint (already exists in api.py) for clinician confirm/override decisions.
- When a clinician overrides a prediction: logs it, flags it as a training sample.
- Periodically triggers: (a) re-ranking of uncertain patients, (b) federated retraining request.
- Tracks model drift: if override rate > 15%, triggers automatic retraining cycle.
- Generates a weekly 'Model Performance Audit' report for the dashboard.

WHY THIS WINS:

- + This closes the feedback loop - CareLink becomes a LEARNING system, not just a static model.
- + The /feedback endpoint already exists - this agent activates it with autonomous intelligence.
- + Active learning is a hot topic in agentic AI - very few hackathon teams implement this.

HACKATHON IMPACT: Turns CareLink from a static predictor into a continuously improving autonomous system.

IMPLEMENTATION SKETCH:

```

class FeedbackLoopAgent:
    DRIFT_THRESHOLD = 0.15 # 15% override rate triggers retraining

    async def monitor_and_act(self):
        stats = await db.get_feedback_stats(days=7)
        override_rate = stats['overrides'] / stats['total']
        if override_rate > self.DRIFT_THRESHOLD:
            await self.trigger_retraining(stats['override_samples'])
            await self.notify_supervisor('Model drift detected')

    async def trigger_retraining(self, samples: list):
        # Start Flower federated retraining round
        await flower_client.initiate_round(new_samples=samples)

```

7

PHI GUARDRAILS AGENT

HIPAA Compliance Filter - Zero PII/PHI in LLM Outputs

WHAT IT DOES:

- Sits between every agent and the LLM - scrubs PHI before sending to Gemini/Claude API.
- Uses regex + NER to detect: patient names, SSN, DOB, MRN, insurance IDs.
- Replaces PHI with anonymized tokens (e.g., [PATIENT_001], [DOB_REDACTED]).
- After LLM returns, de-tokenizes the response for internal use only.
- Logs every scrub action to the audit_log table for compliance evidence.
- Blocks any agent from sending raw patient records to any external API.

WHY THIS WINS:

- + HIPAA compliance in an agentic system is a CRITICAL differentiator for HealthTech.
- + Most hackathon teams send raw patient data to LLMs - this agent proves you do not.
- + CareLink already has differential privacy for FL - this extends privacy to the LLM layer.

HACKATHON IMPACT: Demonstrates enterprise-grade HealthAI responsibility - judges from healthcare sector love this.

IMPLEMENTATION SKETCH:

```
import re
class PHIGuardrailsAgent:
    PHI_PATTERNS = {
        'ssn': r'\d{3}-\d{2}-\d{4}',
        'mrn': r'MRN[:\s]\d+',
        'dob': r'\d{4}-\d{2}-\d{2}',
        'name': r'Patient:\s[A-Z][a-z]+\s[A-Z][a-z]+',
    }
    def scrub(self, text: str) -> tuple[str, dict]:
        token_map = {}
        for phi_type, pattern in self.PHI_PATTERNS.items():
            matches = re.findall(pattern, text)
            for i, match in enumerate(matches):
                token = f'[{phi_type.upper()}]_{i:03d}'
                token_map[token] = match
                text = text.replace(match, token)
        return text, token_map
```

8

RISK ANALYST AGENT

XGBoost + LLM + RAG - Supercharged Readmission Risk Engine

WHAT IT DOES:

- Upgrades the existing /predict/{id} endpoint from a raw ML call to a full agent.
- Step 1: XGBoost predicts probability + SHAP top factors (existing).
- Step 2: RAG Agent retrieves clinical guidelines for each top SHAP factor.
- Step 3: LLM synthesizes a clinician-readable risk narrative with evidence citations.
- Step 4: Compares patient risk to similar historical patients (K-nearest neighbors in Qdrant).
- Output: structured JSON with probability, narrative, citations, similar_cases.

WHY THIS WINS:

- + Existing SHAP explains the model - this agent explains it in clinical language with evidence.
- + The 'similar cases' feature (semantic search in Qdrant) is unique and memorable.
- + RAG-grounded risk narratives prevent pure LLM hallucination about patient risk.

HACKATHON IMPACT: Makes the ML model truly interpretable for clinicians - a core promise of responsible HealthAI.

IMPLEMENTATION SKETCH:

```
class RiskAnalystAgent:
    async def analyze(self, patient_id: int) -> dict:
```

```

# Step 1: Run XGBoost (existing)
ml_result = await predict_api(patient_id)
# Step 2: RAG for each top SHAP feature
evidence = {}
for factor in ml_result['top_factors'][:3]:
    docs = await rag.retrieve(f'readmission risk {factor["feature"]}')}
    evidence[factor['feature']] = docs[0]['text'] if docs else ''
# Step 3: LLM narrative
narrative = await llm.generate(
    f'Patient risk {ml_result["probability"]:.0%}.\n'
    f'Key factors: {ml_result["top_factors"]}.\n'
    f'Evidence: {evidence}. Write a 3-sentence clinical summary.'
)
return {**ml_result, 'narrative': narrative, 'evidence': evidence}

```

9

AGENT OBSERVABILITY LAYER

LangSmith + RAGAS + Prometheus - The Hackathon Differentiator

WHAT IT DOES:

- Every agent call is traced with LangSmith: input, output, latency, token cost.
- RAGAS scores each RAG response: faithfulness, answer_relevancy, context_recall.
- Prometheus exposes /metrics endpoint: agent_calls_total, latency_p95, error_rate.
- A live 'Agent Dashboard' tab in CareLink UI shows real-time agent activity.
- RAGAS faithfulness > 0.85 target (AuRAG achieved > 0.88 - you can match this).

WHY THIS WINS:

- + NO other hackathon team will have agent observability. This is the top-1% differentiator.
- + Shows judges that you think about production readiness, not just demos.
- + RAGAS scores give you quantified proof that your RAG is working - hard numbers win judges.

HACKATHON IMPACT: Turns a demo project into something that looks production-ready. Judges remember this.

IMPLEMENTATION SKETCH:

```

from langsmith import Client as LangSmithClient
import time

class AgentObservability:
    def __init__(self):
        self.ls = LangSmithClient()

    async def trace(self, agent_name: str, fn, *args, **kwargs):
        start = time.time()
        try:
            result = await fn(*args, **kwargs)
            latency = time.time() - start
            self.ls.create_run(name=agent_name,
                              inputs={'args': str(args)},
                              outputs={'result': str(result)},
                              extra={'latency': latency})
            return result
        except Exception as e:
            self.ls.create_run(name=agent_name, error=str(e))
            raise

```

SECTION 4 - 12-Hour Hackathon Implementation Plan

You have 12 hours. Here is the exact order to implement changes so you have a working, impressive demo at hour 10 and two hours for polishing your presentation.

Hour	Task	Output	Priority
0:00-0:30	Setup: Install LangGraph, Qdrant, Sentence-Transformers, Redis	pip install langgraph qdrant-client sentence-transformers redis langsmith	SETUP
0:30-1:30	Build AgentState + Supervisor Node (langgraph StateGraph skeleton)	Working supervisor that routes to 3 placeholder sub-agent nodes	CRITICAL
1:30-3:00	Build Clinical RAG Agent Index 50-100 clinical guideline chunks	Qdrant DB loaded + RAG retrieval working with ClinicalBERT embeddings	CRITICAL
3:00-4:30	Upgrade Triage to Agentic Loop Replace netlify/functions/triage.ts	Python TriageAgent with 3 tools vitals_check + rag + severity_scorer	CRITICAL
4:30-5:30	Build Risk Analyst Agent Wrap existing /predict/{id} endpoint	XGBoost + RAG narrative + citations in one unified agent output	HIGH
5:30-6:30	Build Care Plan Agent Connect to Supervisor output	Auto-generated discharge plan linked to SHAP risk factors	HIGH
6:30-7:00	Build PHI Guardrails Add to every agent's LLM call	PHI scrubber wrapping all LLM calls with audit_log entries	HIGH
7:00-7:30	Build Medication Safety Agent Integrate OpenFDA mock	Drug interaction checker returns safe/unsafe before care plan sends	MEDIUM
7:30-8:00	Add LangSmith tracing Add RAGAS eval on 10 test cases	Agent traces visible in LangSmith UI RAGAS score > 0.80	MEDIUM
8:00-9:00	Frontend: Add 'Agent Cockpit' tab Show live agent activity log	New React tab showing supervisor agent chain, each step's output	MEDIUM
9:00-10:00	Feedback Loop Agent Connect to existing /feedback endpoint	Drift detection + auto-retraining trigger when override rate > 15%	NICE-TO-HAVE
10:00-11:00	End-to-End testing + bug fixes Demo patient flow walkthrough	Full demo: patient > supervisor > triage > care plan > safety check	TESTING
11:00-12:00	Prepare submission deck Record demo video	README, architecture diagram, 2-min Loom demo video	PRESENTATION

SECTION 5 - Complete Tech Stack & pip/npm Requirements

New Python Packages to Add (backend/requirements.txt):

```
langgraph>=0.2.0 # Multi-agent orchestration (Supervisor + state graph)
qdrant-client>=1.9.0 # Vector DB for Clinical RAG
sentence-transformers # ClinicalBERT embeddings for RAG
langsmith>=0.1.0 # Agent tracing & observability
ragas>=0.1.0 # RAG evaluation (faithfulness, relevancy scores)
redis>=5.0.0 # Agent state caching between turns
httpx>=0.27.0 # Async HTTP for OpenFDA drug API
prometheus-client>=0.20 # /metrics endpoint for Prometheus
spacy>=3.7.0 # NER for PHI detection in Guardrails Agent
langchain-google-genai # Upgraded Gemini integration via LangChain
```

New Frontend npm Packages:

```
langsmith # LangSmith frontend SDK for tracing visualization
react-flow-renderer # For visualizing agent graph in the UI
```

New Environment Variables (.env.local):

LANGSMITH_API_KEY	For agent tracing (free tier available)
QDRANT_URL	http://localhost:6333 (local for hackathon)
REDIS_URL	redis://localhost:6379/0
OPENFDA_API_KEY	Free from api.fda.gov - drug interaction checker
GEMINI_API_KEY	Already exists in CareLink
LANGSMITH_PROJECT	carelink-agentic-hackathon

SECTION 6 - Hackathon Judging Scorecard: Before vs. After

Bharat Agentic 2026 judges will evaluate on: Agentic Design, Technical Depth, Real-World Impact, Innovation, and Presentation. Here is your estimated score:

Judging Criterion	Before (Current CareLink)	After (Agentic CareLink)	Score Gain
Agentic Design (30%)	No agents - single LLM call only	9 agents: Supervisor + 7 specialist + Observability	+25 pts
Technical Depth (25%)	Federated ML + SHAP (already strong)	FL + SHAP + RAG + GraphRAG + LangGraph	+15 pts
Real-World Impact (20%)	Risk prediction (static output)	Prediction + Care Plan + Safety + Active Learning	+15 pts
Innovation (15%)	Federated privacy is unique	PHI Guardrails + Agent Observability	+10 pts
Presentation (10%)	Good UI, no agent cockpit	Agent Cockpit tab, LangSmith traces shown	+8 pts
ESTIMATED TOTAL	~45/100	~90/100	TOP-10 / WIN

SECTION 7 - The 2-Minute Demo Script (What to Show Judges)

00:00 - 00:20 - Open CareLink dashboard

Show patients list. Click on a HIGH-risk patient. Say: 'CareLink uses federated learning across 3 hospital nodes to predict 30-day readmission without sharing any patient data.'

00:20 - 00:45 - Trigger the Supervisor Agent

Click 'Run Agentic Analysis'. Show the Agent Cockpit tab lighting up: Supervisor > Triage Agent > Risk Analyst Agent > Care Plan Agent. Say: 'Our LangGraph Supervisor orchestrates 9 specialized agents autonomously.'

00:45 - 01:05 - Show Clinical RAG

In the risk analysis output, point to the cited clinical guidelines. Say: 'Every recommendation is grounded in evidence retrieved from our clinical knowledge base - zero hallucination, full citation.'

01:05 - 01:25 - Show Care Plan + Medication Safety

Show the auto-generated discharge care plan. Then show the medication safety check: 'Our Medication Safety Agent autonomously checked for drug interactions before generating this plan.'

01:25 - 01:45 - Show PHI Guardrails + Observability

Open LangSmith or the Agent Cockpit to show traces. Say: 'Every agent call is traced. PHI is scrubbed before any data leaves the hospital network. We achieved RAGAS faithfulness score 0.85+ on our RAG pipeline.'

01:45 - 02:00 - Close with impact

'CareLink is the only solution today that combines federated privacy, multi-agent orchestration, clinical RAG with evidence grounding, and autonomous care planning in a single HealthTech platform. This is what Agentic AI for Bharat healthcare looks like.'

FINAL SUMMARY - The 3 Things That Will Win You This Hackathon

1. LangGraph Supervisor Agent (MUST HAVE)

This single file transforms CareLink from 'ML project' to 'Agentic AI system'. No Supervisor = Not agentic = Low score. This is non-negotiable.

2. Clinical RAG with Qdrant (MUST HAVE)

Grounded recommendations + cited sources = credibility in HealthTech. Vector DB + ClinicalBERT embeddings + hybrid search. Index clinical guidelines in 30 minutes.

3. PHI Guardrails + Observability (DIFFERENTIATOR)

NO other team will have an agent that scrubs PHI before LLM calls + LangSmith traces. This is what separates top-10 from winner. Show the LangSmith dashboard live.

Best of luck, Nishant! CareLink is already strong - with these 9 layers, it becomes unstoppable.